Course Catalog Data Structures
Pseudocode and Runtime Analysis

Amin Ahmed
CS-300
October 11, 2026

Introduction

For this project I planned out a course catalog program for the CS department that reads courses from a file, checks the file, and stores the courses so an advisor can print every course in order or look up one course with its title and prerequisites, and I wrote pseudocode for three ways to store the courses (a vector, a hash table, and a binary search tree), compared their speed and memory, and picked the best one.

Assumptions

n = number of courses, p = the most prerequisites any course has (a small constant), m = hash table buckets (179).
Course numbers are unique, uppercase, and have the same format (like CS300), so comparing them as text sorts them right.
Each line of pseudocode costs 1, and a line that calls a function costs whatever that function costs.
If anything in the file is wrong, nothing gets loaded.

Course Object and Shared Functions

Each line in the file becomes one Course, and LoadCourses, PrintCourse, and MergeSort are the same for all three structures, and only ClearAll, Insert, FindCourse, ValidatePrerequisites, and PrintAllCourses change, and those are written out in each structure's section.

```
STRUCTURE Course
    courseNumber (string)
    courseTitle (string)
    prerequisites (vector of strings)
END STRUCTURE

FUNCTION LoadCourses(fileName)
    Open the file
    IF the file didn't open
        Display "Could not open the file" and RETURN false
    END IF
    CALL ClearAll()
    WHILE there is another line in the file
        Read the line and split it at each comma into fields
        IF the line is blank, skip it
        IF fields has fewer than 2 items
            Display "Format error: every course needs a number and a title"
            Close the file, CALL ClearAll(), and RETURN false
        END IF
        Create a Course with courseNumber = fields[0] and courseTitle = fields[1]
        Add every field after fields[1] to the course's prerequisites
        CALL Insert(course)
    END WHILE
    Close the file
```

```
    IF ValidatePrerequisites() is false
        CALL ClearAll() and RETURN false
    END IF
    RETURN true
END FUNCTION

FUNCTION PrintCourse(course)
    Display courseNumber + ", " + courseTitle
    Display "Prerequisites: " + the prerequisites separated by commas, or "None"
END FUNCTION

FUNCTION MergeSort(list)
    IF list has 1 course or less
        RETURN list
    END IF
    Set left to MergeSort(first half of list)
    Set right to MergeSort(second half of list)
    Create an empty list called result
    WHILE left and right both still have courses
        Move whichever front course has the smaller courseNumber to the end of result
    END WHILE
    Add whatever is left in left or right to result
    RETURN result
END FUNCTION
```

Menu

```
Set loaded to false
WHILE the user hasn't picked 9
    Display "1) Load Courses", "2) Display All Courses", "3) Find a Course", "9) Exit"
    Read choice
    IF choice is 1
        Ask for the file name, set loaded to LoadCourses(fileName), and say if it worked
    ELSE IF choice is 2 or 3 AND loaded is false
        Display "Please load the courses first"
    ELSE IF choice is 2
        CALL PrintAllCourses()
    ELSE IF choice is 3
        Ask for a course number and make it uppercase
        Set course to FindCourse(courseNumber)
        IF course was found, CALL PrintCourse(course), ELSE display "Course not found"
    ELSE IF choice is not 9
        Display "Not a valid option"
    END IF
END WHILE
Display "Goodbye"
```

Vector

Courses go in a vector called courses in file order, and searching checks them one by one, and printing in order needs a sort first.

```
FUNCTION ClearAll(): clear courses
FUNCTION Insert(course): append course to the end of courses

FUNCTION FindCourse(courseNumber)
    FOR each course in courses
        IF course.courseNumber equals courseNumber, RETURN course
    END FOR
    RETURN nothing
END FUNCTION

FUNCTION ValidatePrerequisites()
    FOR each course in courses
        FOR each prerequisite of the course
            IF FindCourse(prerequisite) finds nothing
                Display "Error: " + prerequisite + " is not in the file" and RETURN false
            END IF
        END FOR
    END FOR
    RETURN true
END FUNCTION

FUNCTION PrintAllCourses()
    Set courses to MergeSort(courses)
    FOR each course in courses
        Display courseNumber + ", " + courseTitle
    END FOR
END FUNCTION
```

Hash Table

Courses go in a hash table with 179 buckets using the course number as the key, and collisions are handled with chaining, where a new course goes at the front of its bucket's linked list so nothing gets overwritten, and since a hash table has no order, printing the full list means gathering and sorting the courses first.

```
STRUCTURE Node
    course (Course)
    next (pointer to the next Node)
END STRUCTURE
Create table with 179 empty buckets

FUNCTION Hash(courseNumber)
    Set total to 0
    FOR each character in courseNumber
        Set total to (total * 31 + the character's code) mod 179
    END FOR
    RETURN total
END FUNCTION
```

```
FUNCTION ClearAll(): delete every node and empty every bucket

FUNCTION Insert(course)
    Set key to Hash(course.courseNumber)
    Put a new node holding course at the front of table[key]'s chain
END FUNCTION

FUNCTION FindCourse(courseNumber)
    FOR each node in the chain at table[Hash(courseNumber)]
        IF the node's courseNumber equals courseNumber, RETURN the node's course
    END FOR
    RETURN nothing
END FUNCTION

FUNCTION ValidatePrerequisites()
    FOR each course in every bucket's chain
        FOR each prerequisite of the course
            IF FindCourse(prerequisite) finds nothing
                Display "Error: " + prerequisite + " is not in the file" and RETURN false
            END IF
        END FOR
    END FOR
    RETURN true
END FUNCTION

FUNCTION PrintAllCourses()
    Copy every course from every bucket's chain into a list
    Set list to MergeSort(list)
    FOR each course in list
        Display courseNumber + ", " + courseTitle
    END FOR
END FUNCTION
```

Binary Search Tree

Courses go in a binary search tree with smaller course numbers on the left and bigger ones on the right, so an in-order traversal prints them in order with no sorting, and the catch is that this tree doesn't balance itself, so if the file was already sorted, the tree would turn into one long line and be as slow as a vector.

```
STRUCTURE TreeNode
    course (Course)
    left, right (pointers to TreeNodes)
END STRUCTURE
Set root to empty
FUNCTION ClearAll(): delete every node and set root to empty

FUNCTION Insert(course)
    IF root is empty, set root to a new node holding course and RETURN
    Set current to root
    LOOP
        IF course.courseNumber < current's courseNumber
            IF current.left is empty, put a new node holding course there and RETURN
            ELSE set current to current.left
```

```
            ELSE
                IF current.right is empty, put a new node holding course there and RETURN
                ELSE set current to current.right
            END IF
        END LOOP
    END FUNCTION

    FUNCTION FindCourse(courseNumber)
        Set current to root
        WHILE current is not empty
            IF current's courseNumber equals courseNumber, RETURN current's course
            Go left if courseNumber is smaller, otherwise go right
        END WHILE
        RETURN nothing
    END FUNCTION

    FUNCTION ValidatePrerequisites()
        FOR each node in the tree (visiting every node once)
            FOR each prerequisite of the course
                IF FindCourse(prerequisite) finds nothing
                    Display "Error: " + prerequisite + " is not in the file" and RETURN false
                END IF
            END FOR
        END FOR
        RETURN true
    END FUNCTION

    FUNCTION PrintAllCourses(): CALL InOrder(root)

    FUNCTION InOrder(node)
        IF node is not empty
            CALL InOrder(node.left)
            Display courseNumber + ", " + courseTitle
            CALL InOrder(node.right)
        END IF
    END FUNCTION
```

Runtime Analysis

This table covers loading a file with n courses and no errors, which is the worst case since an error stops loading early, and all three check prerequisites by searching their own structure for each one, so those searches are counted.

Code	**Line Cost**	**# Times**	**Total Cost**
Same for all three (LoadCourses)			
Open and close the file	1	2	2
Loop through lines: read, split, check blank, check fields	4	n	4n
Create the Course	1	n	n
Add its prerequisites	p	n	np
Subtotal			**np + 5n + 2**

Code	Line Cost	# Times	Total Cost
Vector			
Insert (append to the end)	1	n	n
FOR each course in courses	1	n	n
FOR each prerequisite of the course	1	np	np
FindCourse(prerequisite)	n	np	n^2p
Total			**$n^2p + 2np + 7n + 2 = O(n^2)$**
Hash Table			
Insert (hash, then put at front of chain)	2	n	2n
FOR each course in every bucket's chain	1	m + n	m + n
FOR each prerequisite of the course	1	np	np
FindCourse(prerequisite)	1 avg, n worst	np	np avg, n^2p worst
Total (average)			**$3np + 8n + m + 2 = O(n)$**
Total (worst)			**$n^2p + 2np + 8n + m + 2 = O(n^2)$**
Binary Search Tree			
Insert (walk down the tree)	log n avg, n worst	n	n log n avg, n^2 worst
FOR each node in the tree	1	n	n
FOR each prerequisite of the course	1	np	np
FindCourse(prerequisite)	log n avg, n worst	np	np log n avg, n^2p worst
Total (average)			**$np \log n + n \log n + 2np + 6n + 2 = O(n \log n)$**
Total (worst)			**$n^2p + n^2 + 2np + 6n + 2 = O(n^2)$**

For the vector, the slow part is checking prerequisites, since FindCourse is a linear search that costs n and runs up to np times, so loading is $O(n^2)$, and the hash table's Insert only costs 2 and its FindCourse is O(1) on average, so loading is O(n) on average, but $O(n^2)$ if every course lands in the same bucket, and in the tree, Insert and FindCourse cost as much as the tree is tall, which is about log n for a mixed-up file or n if the file is sorted and the tree becomes a line, and since p is a small constant and m is always 179, they don't change the Big-O.

Operation	Vector	Hash Table	Binary Search Tree
Read and parse the file	O(n)	O(n)	O(n)
Create course objects	O(n)	O(n)	O(n)
Store the courses	O(n)	O(n)	O(n log n) avg, $O(n^2)$ worst
Validate prerequisites	$O(n^2)$	O(n) avg, $O(n^2)$ worst	O(n log n) avg, $O(n^2)$ worst
Total loading	**$O(n^2)$**	**O(n) avg, $O(n^2)$ worst**	**O(n log n) avg, $O(n^2)$ worst**
Find one course	O(n)	O(1) avg, O(n) worst	O(log n) avg, O(n) worst
Print all in order	O(n log n) (sort first)	O(n log n) (gather + sort)	O(n) (in-order)

For finding one course, the vector checks every course, the hash table checks one chain, and the tree follows one path down, and for printing in order, the vector and hash table need merge sort every time (O(n log n)) and the hash table has to gather the courses from all its buckets first, but the tree's in-order traversal is already in order, so it's just O(n).

Memory

	Vector	**Hash Table**	**Binary Search Tree**
The courses	n courses	n courses	n courses
Extra space	Up to n empty spots	179 buckets + 1 pointer each	2 pointers each
Extra for sorting	Merge sort list (n)	Course list + merge sort list (2n)	None
Extra for checking	None	None	Stack as deep as the tree
Total	**O(n)**	**O(n)**	**O(n)**

All three store the same n courses, so they're all O(n) memory, and the difference is the extras, since the vector can have empty spots after it grows, the hash table has buckets and a next pointer per course, and the tree has two pointers per course, and the vector and hash table also need extra lists every time they sort, but the tree doesn't sort at all.

Pros and Cons

Vector: It's the easiest to write and has no pointers, but searching is O(n), loading is $O(n^2)$ because of the prerequisite checks, and printing needs an O(n log n) sort, so it's fine for a small catalog, but it slows down the fastest.

Hash Table: It's the fastest at finding one course (O(1) on average) and at loading (O(n) on average), but it has no order, so every full-list print needs an O(n log n) sort and extra memory, and it's also harder to write because of the hash function and collisions.

Binary Search Tree: It's the only one that's good at both main jobs, since finding a course is O(log n) on average and printing in order is O(n) with no sorting, and memory is O(n) with no wasted space, but the downside is it can turn into a line if the courses come in sorted order, which makes searching O(n).

Recommendation

I'd go with the binary search tree because the program's two main jobs are printing every course in order and looking up one course, and the tree is the only structure that does both well, since it prints the sorted list in O(n) while the vector and hash table need an O(n log n) sort every time, and it finds a course in O(log n) on average compared to the vector's O(n), and even though the hash table is a little faster for one lookup, advisors will print the full list a lot and it has to sort everything each time, and the tree also uses O(n) memory and isn't much harder to write, and its one weakness is getting unbalanced if the courses come in sorted order, but a course file usually isn't sorted, and a self-balancing tree could fix that if the catalog ever got really big.